

The LLMHub GPU Fit Estimator: Memory Model, Launch Gate, and Production Hardening

LLMHub / SPIN

August 14, 2026

Abstract

LLMHub now estimates whether a vLLM deployment will fit on NCSA Delta GPUs before submitting it to Slurm, and blocks launches that provably cannot boot. This note explains the memory model and its calibration, the launch-gate contract, the five production defects found during pre-release review (two false-accept channels, three availability channels), the fixes, and the remaining caveats the team should know about.

1 Two verdicts, one model

Everything in the estimator rests on distinguishing two questions that sound alike but have very different answers:

“**Does it fit?**” (**fits**) Could the GPU hold the *worst case* — weights, plus KV cache for `max_num_seqs` concurrent sequences each at full context, plus overhead? This is a planning number shown in the UI survey. vLLM never actually reserves this much.

“**Does it boot?**” (**starts**) After loading weights and overhead, is the leftover KV pool big enough to hold *one* full-length sequence? This is the check vLLM itself performs at startup, and it is what the launch gate enforces. Beyond the pool, vLLM queues and preempts requests — it does not OOM — so sustained concurrency is reported as capacity, never gated.

`fits=false` with `starts=true` is the *normal* operating regime, not an edge case: Qwen2.5-7B on one A40 at context 4096 and concurrency 256 “needs” a fictional 76 GiB but boots with a real 24.9 GiB KV pool ($\approx 466k$ tokens, ≈ 113 concurrent full-context sequences).

2 The memory model

All quantities are per GPU, in binary GiB (2^{30} bytes). Three additive terms are compared against measured VRAM:

$$\underbrace{\frac{W}{t}}_{\text{weights}} + \underbrace{n_{\text{tok}} \cdot k_{\text{tok}}(t)}_{\text{KV cache}} + \underbrace{(1-u)V + 0.8 + 0.002m + 0.25[t > 1]}_{\text{overhead}} \leq V \quad (1)$$

where W is the model’s weight size (read from safetensors metadata, not computed), t the tensor-parallel size, $u = 0.9$ the vLLM memory-utilization fraction, V the measured per-GPU VRAM, m the scheduler concurrency (`--max-num-seqs`), and the per-token KV cost is

$$k_{\text{tok}}(t) = 2 \cdot L \cdot \max\left(1, \frac{H_{kv}}{t}\right) \cdot d \cdot b, \quad (2)$$

with L layers, H_{kv} KV heads, d head dimension, and b bytes per element. The $\max(1, \cdot)$ term captures vLLM’s KV-head *replication*: when $H_{kv} < t$ (common for GQA at high TP), per-GPU KV stops shrinking. Weights shard by t ; the utilization reserve and internal overhead are paid in full by *every* rank.

2.1 Calibration

The internal-overhead line $0.8 + 0.002m$ was fitted as a conservative upper bound over eight historical vLLM 0.11.0 startup probes on Delta (a ninth, captured live from the production launch path, validates it — §6) (Qwen2.5-0.5B/7B; A40 and A100-40GB; TP 1/2/4; $m \in \{32, 256, 512, 1024\}$), reading vLLM’s own “Model loading took” and “Available KV cache memory” log lines against `nvidia-smi` totals. The model under-promises the measured KV pool at every probe point:

Probe	m	Predicted pool	Measured	Margin
0.5B / A40 / TP1	256	38.25	38.51	+0.26
7B / A40 / TP1	256	24.93	25.17	+0.24
7B / A40 / TP2	256	31.81	32.26	+0.45
7B / A40 / TP4	256	35.37	35.85	+0.48
7B / A100 / TP1	256	20.44	20.73	+0.29
7B / A40 / TP1	32	25.38	25.43	+0.05
7B / A40 / TP1	512	24.42	24.66	+0.24
7B / A40 / TP1	1024	23.39	23.64	+0.25

A regression test asserts `predicted ≤ measured` with zero slack (safety: an optimistic model would let the gate approve launches that OOM) plus an accuracy bound of 1.2 GiB. VRAM values in the hardware table are `nvidia-smi` measurements, not marketing numbers (A40: 44.988, not 48).

3 The launch gate

Immediately before `vec-inf` submits a Slurm job, the gate resolves the launch tuple — HF model id, partition, context, TP, concurrency — with the precedence that actually reaches vLLM: user free-form `vllm_args` > explicit request fields > curated catalog (tensor-parallel size is the exception — the launch path never emits it from the request’s GPU count, so the catalog flag wins there). It then certifies the boot contract:

$$\frac{W}{t} + C \cdot k_{\text{tok}}(t) + \text{overhead}(m_{\text{launch}}) \leq V, \quad (3)$$

where C is the resolved `max_model_len` and, crucially, *the KV budget is one sequence but the overhead is charged at the concurrency the job will actually boot with* ($m_{\text{launch}} = \text{user override} > \text{catalog} > \text{the vLLM default} = \mathbf{1024}$ on the V1 engine in the production container, not the V0-era 256; see §6).

The gate **blocks** only on a real sizing verdict: the pool cannot hold one full-context sequence, or TP exceeds the partition’s GPU count. It **skips with a warning** — launch proceeds, auditable in logs — whenever it cannot give an honest verdict: partition not in the Delta hardware table, multi-node launches (not modeled), non-NVIDIA GPUs, unresolvable model metadata (sparse VLM configs, HF outage, gated repo), or a failed catalog lookup. The public `POST /api/validate-config` endpoint remains strictly fail-closed (“cannot verify” is a refusal there); the skip semantics apply

only to the launch gate, where blocking a curated model that launched fine yesterday would be the worse failure. An estimator crash anywhere in this path is caught and logged; it never blocks a launch or leaks the GPU allocation.

4 What pre-release review found and fixed

4.1 False-accept: overhead charged at the wrong concurrency

The gate originally charged overhead at $m = 1$ while 90 of 94 catalog entries boot at the vLLM default $m = 256$. The $0.002 \cdot 255 \approx 0.51$ GiB gap exceeds every calibration margin (0.24–0.48 GiB at $m = 256$), so a genuine false-accept window existed. Concretely, for Llama-3.3-70B on `gpuA100x4` (TP=4): the $m=1$ contract certifies a pool of $\approx 27,400$ tokens, while the calibrated pool at $m=256$ holds $\approx 20,800$ — a launch at context 24,102 passed the gate and died at boot. **Fix:** the overhead term now uses the resolved launch concurrency (regression test pins the exact scenario above). At user-set $m=512/1024$ the closed gap would have been 1.0/1.8 GiB.

4.2 False-accept: unpinned context certified at an invented 4096

If neither the request nor the catalog pinned `--max-model-len`, the gate certified context 4096 — but vLLM boots at the model’s *native* `max_position_embeddings` (e.g. 131k), so the certificate was for a contract weaker than reality. **Fix:** an unpinned context now validates against the native value. Related: user free-form `vllm_args` (which reach vLLM *last* and win) were invisible to the gate and could raise the context after certification; the gate now parses them at highest precedence.

4.3 Availability: ~ 20 curated VLM models were unlaunchable

VLM configs nest the language model’s dimensions under `text_config/llm_config/language_config`; the metadata mapper read only top-level keys, so every such model resolved to “cannot verify” and was *hard-blocked* — a regression for models that launch fine today. **Fix (two parts):** the mapper now merges the nested language-model sub-config (verified against real InternVL2.5 and Llama-3.2-Vision-shaped configs), and verdicts that mean “we cannot model this” are flagged `unverifiable` and *skip* the gate instead of blocking. Models whose configs are irreducibly sparse (llava-1.5’s `text_config` relies on transformers class defaults we refuse to replicate; deepseek-vl2; glm-4v; Pixtral has no `config.json`) launch unvalidated with a logged warning — exactly the pre-gate behavior.

4.4 Availability: multi-node entries were hard-blocked

Eight curated entries (Llama-405B, Mistral-Large, Mixtral-8x22B, Command-R+, Llama-3.2-90B-Vision) have `num_nodes > 1` and were unconditionally rejected because multi-node memory math is out of scope. **Fix:** multi-node launches skip the gate with a warning. (Policy note: the team may later prefer advisory warnings in the UI; the math for pipeline parallelism remains future work.)

4.5 Second review round: adversarial findings

An adversarial pass over the fixed code (trying to *break* the gate) surfaced and closed a further set of defects:

- **Comma-joined flag strings.** LLMHub’s wire format joins vLLM flags with commas (`--max-model-len=131072`) — the flag parser was whitespace-delimited, so only the *last* flag of a multi-flag string parsed — re-opening the user-override false accept for every earlier flag. The parser now splits on both.
- **Model-identity steering.** The request’s `hf_model` field steered gate sizing, but `vec-inf` launches the *catalog* model’s local weights when they exist — a tiny stand-in repo could earn a confident `valid=True` for a 70B launch. Sizing now follows the catalog-derived identity whenever the two disagree (logged).
- **Unmodeled memory flags.** Free-form flags that change vLLM’s memory picture (`--gpu-memory-utilization`, `--dtype`, `--kv-cache-dtype`, LoRA/speculative flags, ...) now cause a loud skip: certifying a config whose memory behavior we did not model would be a guess dressed up as a verdict.
- **Dequantized-at-load checkpoints.** MXFP4/FP8 checkpoints (e.g. `gpt-oss-120b`) have no native kernels on Ampere; vLLM materializes bf16 weights 2–4× the on-disk size we measure. On A40/A100 such models are now refused as unverifiable (skip) instead of certified at disk size and OOMing at load. On H200 they size normally.
- **MLA models.** DeepSeek-V2/V3-family configs (compressed latent KV) are over-counted ~10–25× by the standard formula — a confident wrong verdict in either direction — and are now refused as unverifiable.
- **Crash-to-bypass hardening.** A crash inside the gate skips it (by design), so crashing must never be the cheap way past: concurrency overrides are clamped to ≥ 1 , all catalog/flag integer coercions are guarded (a YAML `--max-model-len: yes` no longer certifies a context of one token), and mixed-provenance VLM configs (top-level vision dims combined with partial text dims) can no longer produce a confidently wrong `head_dim`.
- **Launch-path regression (pre-existing).** `num_nodes` always arrives as an explicit `None` from the request schema, so the old dict-get default never fired and `num_gpus × num_nodes` raised `TypeError` on every dialog launch that set a GPU count. Fixed and covered by new service-level tests.

The same pass produced strong negative evidence: the resolved-concurrency threading is correct end to end, the UI’s `starts` and the gate’s verdict were shown *algebraically identical* and agree on a 900-point sweep plus a boundary-crossing regression test, TP edge cases (replication floor, non-divisible heads, partition caps) could not be made optimistic, and the streamed header reads could not be made to buffer unbounded data.

4.6 Robustness: crash paths, HF coupling, H200

The gate ran up to four sequential HTTPS calls to Hugging Face per launch (after GPU allocation, uncached), an unexpected estimator exception would surface as a 500 *and leak the GPU-allocation counter*, and a range-ignoring mirror could stream a multi-GB weights file into backend memory. **Fixes:** successful metadata lookups are cached (10-minute TTL, bounded size; failures are never cached), the gate call is wrapped fail-open with the exception logged, safetensors header reads are streamed with hard byte limits, malformed headers degrade to “unknown weights” (never a partial undercount, which would be optimistic), and 401/403 from HF now produce actionable messages (missing token vs. unaccepted license). H200 rows carried the nominal 141 GiB although H200s report ≈ 140.4 GiB in `nvidia-smi`; the table now carries a padded educated guess of **140.0 GiB** until probes run on real hardware.

5 Caveats and known limitations

- **H200 is an educated guess.** Neither its VRAM nor the internal-overhead line has been probed there (0.4 GiB pad applied); same for $TP \geq 8$ (the 0.25 GiB TP buffer is a safety margin, measured ≈ 0 at $TP \leq 4$). First H200/TP-8 probes should tighten both.
- **The Option-B surface is intentional.** The gate certifies boot, not saturated concurrency. A booted model under extreme burst degrades by queueing/preemption, not OOM; capacity numbers in the UI are advisory.
- **Unverifiable models launch unvalidated** (llava-1.5/1.6, deepseek-vl2, glm-4v, Pixtral, plus anything during an HF outage). This is a deliberate availability-over-strictness call for the *launch gate* only; every skip is logged. The team should confirm this policy.
- **Workload archetype factors are placeholders** (marked as such in the data file); they affect only the advisory “typical” column, never gating.
- **Architectures the formula over-estimates:** MLA models (DeepSeek-V2/V3 compress KV; our formula can over-charge them $\sim 25\times$ — they are multi-node anyway, hence skipped) and sliding-window/hybrid-attention models (Mistral, Gemma-2/3) where newer vLLM allocates less than full KV per layer. Both err conservative. FP8-KV runs are likewise over-charged $2\times$ (we use the weight dtype for KV). None of these can cause a false accept.
- **SU rate anomaly:** `gpuA100x8-interactive` is tabled at 3000 SU/GPU-hr while every other interactive partition is exactly $2\times$ its batch rate (which would give 2000). Cost display only — gating is unaffected — but worth confirming against Delta billing.
- **Calibration breadth:** nine probes, two models ($\leq 7B$), two GPU types, one vLLM version (0.11.0). The conservative-line + zero-slack regression test protects against silent drift, but two extrapolations are load-bearing and unprobed: 70B-class verdicts (vLLM’s profile-run peak scales with `hidden_size`, which the flat overhead line measured on 0.5B/7B does not capture — some passing 70B configs sit within ~ 1 GiB of the line) and any vLLM upgrade on Delta (the overhead line is version-dependent). A 70B/H200/TP-8 probe round is the single highest value follow-up.
- **The two API surfaces are deliberately different.** The launch gate certifies boot; `POST /api/validate-config` is a strict worst-case survey (KV at $256\times$ context) and will say `valid:false` for many configs that boot and launch fine. Responses now carry `unverifiable` so callers can tell “will not fit” from “cannot model it (and the launcher would proceed)”. The survey’s `fits` field is the same fictional worst case — UI and tooling should read `starts`.
- **Operational prerequisites.** `HF_TOKEN` must be set in the production backend environment: without it every gated repo (`meta-llama/*`, `google/*`, some `mistralai/*` — roughly half the catalog) resolves as unverifiable and launches *ungated*. Skips are logged at WARNING; a periodic check of skip rate and skip reasons is the cheapest way to notice the gate silently losing coverage.

6 Live end-to-end validation on Delta

The full path was exercised live from a Delta login node against a throwaway local database (the shared database was never touched):

- **Gate-block, live:** a deliberate over-budget request (Qwen2.5-7B at context 500,000 on one A40) returned HTTP 400 “Launch blocked: ... over by 1.7 GiB” with the split-contract breakdown, released the GPU allocation, and submitted no Slurm job.
- **Launch, live:** Qwen2.5-0.5B-Instruct through `POST /api/models/deployments` → gate pass → vec-inf → Slurm job 21145772 → *vLLM booted* (“Application startup complete”) on `gpuA40x4`; the job was then cancelled.
- **Model validated against the boot log:** weights loaded at 0.9271 GiB (fixture: 0.9267); the KV pool was 3,288,608 tokens at 37.64 GiB — *exactly* 12,288 bytes/token, matching the formula to the byte.
- **One real defect found and fixed:** the job booted with `max_num_seqs = 1024` — the *vLLM V1-engine* default — while the resolver assumed the V0-era 256. Measured internal overhead was 1.92 GiB vs. the 1.31 GiB the gate would have charged at 256 (~0.6 GiB optimistic); at the correct 1024 the model charges 2.85 GiB (conservative by 0.93 GiB). `DEFAULT_MAX_NUM_SEQS` is now 1024, and the live measurement is vendored as a ninth calibration probe so regression tests pin it. (The frontend’s copy of the constant is updated in the same PR.)

7 Portability to other Slurm clusters

`python -m app.services.fit_estimator.discovery --output hardware.yaml` reads partitions, GPU GRES names, and walltimes from `sinfo` and emits a hardware table in the estimator’s schema; `FIT_ESTIMATOR_HARDWARE_YAML` activates it. VRAM resolves by (1) `--probe` (a tiny `srun nvidia-smi` per GPU type — measured truth) or (2) a bundled table that deliberately assumes the *smallest* memory variant for ambiguous GRES names, so an unprobed cluster can only produce false rejects. Unknown GPUs are excluded, never guessed. SU rates are site policy and must be hand-edited; the overhead calibration transfers as-is (it is a property of the *vLLM* version, not the GPU). Validated by regenerating Delta’s own table from live `sinfo` and reproducing identical verdicts.

8 Verification status

158 backend tests pass (including the live production-container probe and the Slurm hardware-discovery suite) (fully offline; real HF configs and the eight probe measurements are vendored as fixtures), including regression tests for the boot-window false accept, comma-joined flag parsing, model-identity protection, the skip semantics, nested-config resolution and its mixed-provenance guard, MLA/dequantization refusals, metadata-cache behavior (TTL, no negative caching, bounded eviction), malformed-safetensors handling, survey-vs-gate agreement across the boot boundary, and the `launch_model` resource-accounting contract (release on block, fail-open on crash, the `num_nodes` regression). The gate-pass/boot-fail scenario, user-override visibility, native-context fallback, and both skip paths were additionally exercised end-to-end against the real gate code with only the network fetch stubbed, and two independent adversarial review passes were run over the final code.